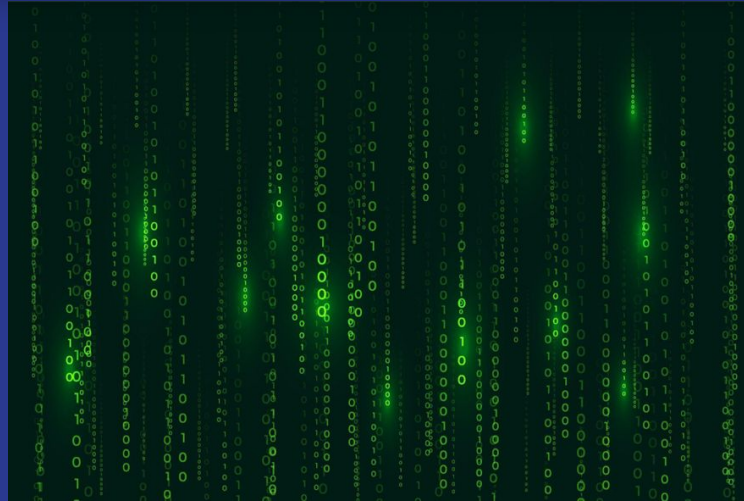


MÓDULO I

Introdução à Programação II



A Educação é o primeiro passo para um futuro melhor...

MÓDULO I

Quais são os paradigmas de programação mais importantes?

Embora existam algumas similaridades importantes entre as linguagens e os **paradigmas de programação**, esses dois conceitos (que ainda geram certa confusão) são bem diferentes entre si.

E é justamente sobre isso que vamos falar hoje.

Enquanto as **linguagens de programação** são meios utilizados para instruir e comunicar os computadores a fazerem diferentes tarefas e ações, os paradigmas funcionam como uma espécie de identidade dessas linguagens.

São eles que expressam, a partir de um conjunto de características, como uma linguagem funciona e soluciona problemas – o que contribui significativamente para a construção de códigos mais legíveis e organizados.

MÓDULO I

O que é um paradigma de programação

Os *paradigmas de programação* são, antes de tudo, um meio de qualificar a linguagem com base em sua funcionalidade.

Em outras palavras, eles podem ser entendidos como um *estilo*, *modelo* ou *metodologia* de programação, que *apontam para a melhor forma de solucionar problemas usando uma determinada linguagem*.

O que acontece é o seguinte: com tantas linguagens de programação existentes, é importante que elas sigam algumas regras na hora de serem implementadas. São essas regras, por sua vez, que ficaram conhecidas como *paradigmas*.

Da mesma forma, quando uma nova linguagem de programação é desenvolvida, ela tende a se enquadrar em um *paradigma* ou até mesmo em mais de um, conforme suas peculiaridades.

MÓDULO I

Esse é um assunto que pode parecer um pouco complicado para quem está no começo da jornada como desenvolvedor...



A Educação é o primeiro passo para um futuro melhor...

MÓDULO I

Por que é importante aprender sobre paradigmas de programação

Os paradigmas também oferecem as técnicas mais apropriadas para cada tipo de aplicação.

O que, por sua vez, traz mais produtividade para o dia a dia do desenvolvedor.

Sendo assim, **quando um desenvolvedor domina esse conceito ele automaticamente se transforma em um profissional melhor**, que é capaz de entender as linguagens de forma mais ampla e até mesmo ler nas entrelinhas dos códigos.

Outro ponto relevante é que o conhecimento sobre os paradigmas certamente adicionará uma vantagem competitiva para o seu perfil numa seletiva de emprego.

Quando recrutadores ouvem que você domina o assunto, isso agrega ainda mais valor ao seu nome durante o processo e gera mais confiança na sua contratação!

MÓDULO I

Quais são os tipos de paradigmas de programação

A verdade é que existem inúmeras opções nesta frente. Por isso, vamos destacar os mais importantes levando em consideração dois grupos principais: **os imperativos e os declarativos**.

Paradigmas imperativos

Os **paradigmas de programação imperativo** são focados em instruções exatas que devem ser passadas ao computador na sequência em que serão executadas.

Aqui, basicamente, **o programador instrui a máquina sobre como devem ser computados os processos**, em uma espécie de passo a passo detalhado dos procedimentos.

Entre as vantagens dos paradigmas que pertencem a esse grupo estão a eficiência e flexibilidade, além da possibilidade de permitir uma modelagem tal qual o mundo real.

No entanto, trata-se de um paradigma relativamente complexo e, por isso, ele é mais indicado na construção de aplicações que não demandam manutenção no curto prazo ou mudanças muito frequentes.

Dentro dos paradigmas de programação do tipo imperativo, estão os seguintes:

MÓDULO I

Programação Procedural

Muitas vezes usada como sinônimo de paradigma imperativo, a programação procedural é excelente para programação de uso geral e consiste numa lista de instruções para informar ao computador o que fazer.

No geral, esse paradigma determina que as instruções a serem passadas ao computador podem ser agrupadas em procedimentos – que, por sua vez, visam a reutilização do código em pontos diferentes do mesmo

A maioria das linguagens de programação ensinadas na faculdade são procedurais, como:

- **C, C++, Java, Pascal**

No geral, as linguagens que se enquadram nessa categoria são mais indicadas nas seguintes situações:

- Quando existir uma operação complexa que inclui dependências entre operações e quando há necessidade de visibilidade clara dos diferentes estados do aplicativo;
- O programa é muito único e poucos elementos foram compartilhados;
- O programa é estático e não se espera que mude muito ao longo do tempo;
- Espera-se que nenhum ou apenas alguns recursos sejam adicionados ao projeto ao longo do tempo.

MÓDULO I

Programação orientada ao objeto

A programação orientada a objetos (OOP) está entre os paradigmas de programação mais populares do mundo. Isso se deve, principalmente, aos seus inúmeros benefícios, como a modularidade do código e a capacidade de associar diretamente problemas reais em termos de código.

Além disso, ele foi o primeiro paradigma a permitir a programação multiplataforma.

Uma das grandes preocupações do OOP é realçar o que é realmente importante.

Não por acaso, ele surgiu com o objetivo de permitir o desenvolvimento mais ágil de programas, com maior confiabilidade e redução de custos.

Neste caso, todos os objetos possuem determinados estados e comportamentos. Enquanto os estados são descritos pelas classes como atributos, a forma como eles se comportam é definida por meio de métodos.

MÓDULO I

Entre as principais linguagens que o implementam, estão:

- **PHP**
- **Java**
- **Ruby**
- **C#**
- **Python**

No geral, vale a pena utilizar os **paradigmas de programação orientada a objetos** quando:

- Vários programadores atuam juntos e não precisam entender tudo sobre cada componente;
- Existe muito código a ser compartilhado e reutilizado;
- São previstas muitas mudanças no projeto.

MÓDULO I

Computação paralela

Para completar a lista dos paradigmas imperativos, temos o paradigma de computação paralela. Um sistema de computação paralela permite que muitos processadores executem um programa em menos tempo, dividindo-os.

Trata-se, portanto, de uma forma de resolução de problemas na qual vários computadores trabalham simultaneamente para chegar a um mesmo objetivo.

Entre as linguagens que suportam essa abordagem, destacam-se:

- C, C++

Além disso, o paradigma de computação paralela geralmente é recomendado quando:

- Você tem um sistema que possui mais de uma CPU ou processadores multinúcleo;
- É preciso resolver problemas computacionais que podem levar até dias para serem resolvidos;
- Se trabalha com simulação computacional, inteligência artificial ou modelagem que exige muitos cálculos dinâmicos.

MÓDULO I

Paradigmas de programação declarativos

Agora que já explicamos o que são os **paradigmas imperativos**, vamos para o segundo grupo: os **paradigmas declarativos**!

Neste caso, o programador apenas declara as propriedades do resultado desejado, mas não informa a máquina sobre como devem ser feitos os cálculos relacionados.

Em outras palavras, os **paradigmas declarativos** focam mais no “quê” deve ser resolvido e não em “como” fazê-lo.

Ao utilizá-los, portanto, o desenvolvedor deve declarar verdades lógicas imutáveis, para as quais os resultados serão sempre os mesmos.

Entre as principais vantagens associadas a essa categoria estão a facilidade de acesso a banco de dados e o maior nível de abstração do código.

Além disso, os programas feitos com uma linguagem declarativa costumam ser menores, já que é preciso usar menos código para realizar um objetivo

No grupo dos paradigmas de **programação declarativos** estão:

MÓDULO I

Paradigma de lógica de programação

O paradigma da programação com apontamento lógico não é composto de instruções e, por isso, se difere bastante dos demais (apesar de derivar do declarativo).

Ele é baseado em fatos e usa tudo o que sabe para criar um cenário onde todos esses fatos e cláusulas são verdadeiros e apontam para algum final.

Por exemplo: o *JavaScript* é uma linguagem de programação, todas as linguagens de programação são importantes e, por dedução lógica, *JavaScript* é importante.

Por obter resultados através do raciocínio lógico-matemático, ele é mais popular entre quem trabalha com Inteligência Artificial. Mas também pode ser usado com sucesso em projetos de comprovação de teoremas e na criação de programas especialistas.

Entre as linguagens de programação que utilizam esse paradigma, estão:

- **Absys, Ciao, Alice, Mercury, Prolog**

MÓDULO I

Paradigma funcional

Considerado uma das derivações mais famosas do **paradigma declarativo**, o **paradigma funcional** recebe esse nome por se basear no uso de funções matemáticas.

Neste caso, o programa é composto de funções curtas, no qual todo o código está dentro de uma função e todas as variáveis têm escopo definido para a função.

No **paradigma de programação funcional**, as *funções* não modificam nenhum valor fora do escopo dessa *função* e as próprias *funções* não são afetadas por nenhum valor fora do escopo.

Aqui, subdivide-se o problema proposto e as *funções* implementadas ficam responsáveis por fazer os cálculos matemáticos. Sendo assim, o **paradigma funcional** é bastante indicado nos casos em que há matemática envolvida diretamente na programação.

São exemplos de linguagens que usam este **paradigma**:

- **Haskell, Scala, Racket, JavaScript**

MÓDULO I

O que é uma linguagem de programação?

Uma *linguagem de programação* é como um conjunto de *regras* e de *instruções* que vão permitir que você comunique as suas ideias e intenções a um computador. Afinal, um programador precisa entender a "*linguagem da máquina*" para criar **softwares**, **aplicativos** e várias outras soluções tecnológicas.

Você já parou para pensar, por exemplo, em como nós — humanos — nos comunicamos com os nossos amigos, familiares e colegas de trabalho?

Utilizamos palavras, expressões e até mesmo a linguagem corporal para interagirmos, né?!

Agora, quando o assunto envolve se comunicar com uma máquina, como os computadores, a pergunta-chave é: Como fazê-lo? Bem, é aí que entra o incrível mundo das linguagens de programação.

Imagine um cenário em que estivéssemos ensinando uma linguagem específica para que a máquina se tornasse capaz de executar determinadas tarefas de acordo com as nossas necessidades. Já pensou em ter a habilidade de "conversar" com o seu computador e de dizer a ele exatamente o que você quer que seja feito? Pois é! Isso só é possível graças às linguagens de programação. Inclusive, o mais interessante é que existem várias delas — cada uma com as suas próprias características e vantagens. Algumas são mais adequadas a certos tipos de projetos; enquanto outras ganham relevância pela facilidade de aprendizado.

MÓDULO I

Quais são os tipos de linguagem de programação?

As linguagens de programação são classificadas em dois tipos principais:

As linguagens de baixo nível e as linguagens de alto nível.

A seguir, vamos entender como funcionam?! Confira!

1. **Linguagens de programação de baixo nível**
2. **Linguagens de programação de alto nível**

MÓDULO I

1. Linguagens de programação de baixo nível

Essas linguagens são orientadas à máquina, como uma espécie de conexão entre o hardware e o software. Basicamente, elas viabilizam um controle direto sobre o equipamento e sobre a sua estrutura física, exigindo que o programador tenha um conhecimento bastante aprofundado do hardware.

A categoria se divide em dois tipos diferentes, como vamos ver a seguir.

a. Linguagem de máquina

É a forma mais primitiva de linguagem, sendo composta por dígitos ou bits binários (**0 e 1**) que o computador lê e consegue interpretar. Inclusive, trata-se do único idioma que as máquinas entendem diretamente.

b. Linguagem Assembly

É uma tentativa de substituir a linguagem de **máquina** por algo mais próximo da linguagem humana. Os programas desenvolvidos na linguagem **Assembly** são armazenados como textos e, geralmente, são instruções que correspondem às ações executáveis por um microprocessador.

Esse tipo é mais legível, mas ainda precisa ser convertido para a linguagem de máquina pelo **Assembler**. Afinal, o equipamento não consegue entendê-la.

MÓDULO I

2. Linguagens de programação de alto nível

São linguagens que buscam facilitar o trabalho do programador, usando instruções mais compreensíveis.

Além disso, elas permitem que você escreva códigos com a utilização de idiomas humanos (*português, inglês, espanhol etc.*), que serão traduzidos para a linguagem da máquina por tradutores.

a. Tradutor

Traduz os programas desenvolvidos em uma linguagem de programação para a linguagem de máquina enquanto são executados. Via de regra, há uma grande agilidade na tradução, porém, a atividade deve ser contínua durante a execução.

b. Compilador

Possibilita a tradução completa de um programa de uma só vez, tornando-o mais rápido. Aliás, o código compilado pode ser armazenado para uso futuro, sem que uma nova tradução seja necessária. Logo, temos mais eficiência na execução e no armazenamento do código compilado para eventuais utilizações em outros momentos.

MÓDULO I

Qual é a importância de conhecer as principais linguagens de programação?

A verdade é que essa é uma questão que abre portas para grandes oportunidades e vantagens no mundo do desenvolvimento de softwares.

A propósito, elencamos alguns motivos pelos quais vale (e muito!) a pena conhecê-las. Veja!

- Ampliação da versatilidade profissional
- Viabilização da execução de uma grande gama de projetos
- Aumento da compreensão dos códigos existentes
- Facilitação do aprendizado
- Expansão da empregabilidade
- Participação em comunidades e projetos open source

MÓDULO I

1. Ampliação da versatilidade profissional

As principais linguagens de programação são muito usadas em vários setores e projetos. Portanto, ao conhecê-las, você se torna um profissional mais versátil, capaz de adaptar as suas habilidades a diferentes demandas do seu mercado de atuação.

2. Viabilização da execução de uma grande gama de projetos

Como dito, cada linguagem tem as suas próprias características e é mais — ou menos — adequada para determinados tipos de projetos. Então, conhecer as principais permite que você escolha a mais apropriada para os requisitos da demanda em questão.

3. Aumento da compreensão dos códigos existentes

Muitos projetos já utilizam códigos escritos em linguagens de programação específicas. Diante disso, conhecer as principais faz com que você tenha a capacidade de entender e, inclusive, de colaborar em atividades preexistentes, acelerando processos de manutenção e de desenvolvimento, viu?!

MÓDULO I

4. Facilitação do aprendizado

Não há como negar: conhecendo as linguagens de programação principais, o aprendizado de novas passa a ser mais fácil. Muitos conceitos e estruturas são compartilhados entre elas, o que possibilita uma transição suave para novos ambientes de programação.

5. Expansão da empregabilidade

Atualmente, as empresas frequentemente buscam profissionais com conhecimento em linguagens amplamente adotadas. Logo, ter experiência na utilização das principais delas pode aumentar expressivamente as suas chances de conseguir boas oportunidades e de evoluir profissionalmente.

6. Participação em comunidades e projetos open source

É interessante ressaltar que as principais linguagens são utilizadas em projetos de código aberto e em comunidades de desenvolvedores. Nesse contexto, ao conhecê-las, você pode contribuir ativamente em determinadas atividades, expandir a sua rede profissional e, claro, aprender com outros especialistas.

Sem dúvida alguma, estar por dentro das principais linguagens de programação é um grande diferencial. Na verdade, mais do que isso, trata-se de uma habilidade indispensável para quem busca se destacar nesse campo do desenvolvimento de softwares, que é tão competitivo.

MÓDULO I

Quais são as 10 principais linguagens de programação?

- **Python**
- **C#**
- **C++**
- **JavaScript**
- **PHP**
- **Swift**
- **Java**
- **Go**
- **SQL**
- **Ruby**

É chegada a hora de ver de perto as 10 principais linguagens de programação mais utilizadas. Veja também alguns insights valiosos para quem busca aprimorar as suas habilidades em programação web.

MÓDULO I

1. Python

A **Python** é uma linguagem de alto nível e propósito geral, destacando-se pela sua versatilidade.

Ela é usada em diversos processos — desde a análise e a visualização de dados até o desenvolvimento web.

Aliás, a linguagem passou a ser uma escolha popular para várias tarefas. Veja algumas das suas características:

- *suporte a múltiplos paradigmas de programação;*
- *alta eficiência como linguagem de scripts;*
- *código aberto, permitindo modificações conforme necessário;*
- *comunidade ativa e vasta biblioteca padrão;*
- *integração fácil a outras linguagens.*

MÓDULO I

2. C#

A **C#**, orientada a objetos e integrante da plataforma .NET da Microsoft, é muito empregada no desenvolvimento de aplicativos Windows e web. Algumas das suas características incluem:

- *facilitação da criação de softwares modulares;*
- *integração sólida com tecnologias Microsoft;*
- *suporte robusto para o desenvolvimento de jogos;*
- *viabilização de uso em ambientes corporativos para aplicações críticas.*

3. C++

A **C++** acaba se destacando por sua eficiência e pelo seu controle próximo ao hardware, sendo uma escolha comum para o desenvolvimento de sistemas e de jogos.

A seguir, confira algumas das suas características mais marcantes:

- *eficiência em termos de recursos;*
- *controle preciso sobre a manipulação de memória;*
- *grande uso em sistemas embarcados;*
- *base para muitas outras linguagens de programação.*

MÓDULO I

4. JavaScript

A **JavaScript** é uma linguagem padrão para o desenvolvimento web, viabilizando a criação de interfaces interativas e dinâmicas, sendo executada no navegador do usuário. Algumas das suas características abrangem:

- *grande utilização no desenvolvimento front-end;*
- *tecnologia moderna, representando uma boa linguagem de programação para iniciantes;*
- *suporte à programação assíncrona;*
- *ecossistema robusto de bibliotecas e frameworks.*

5. PHP

A **PHP**, uma linguagem de script para o desenvolvimento web do lado do servidor, destaca-se por sua especialização em web dinâmico e pelo suporte extensivo a bancos de dados. Algumas das suas características principais incluem:

- *suporte ao desenvolvimento web dinâmico;*
- *utilização global e código aberto;*
- *integração fácil a diferentes bancos de dados;*
- *ampla adoção em sistemas de gerenciamento de conteúdo.*

MÓDULO I

6. Swift

A **Swift** — a linguagem da Apple — é projetada para criar aplicativos iOS e macOS, destacando-se por sua sintaxe clara e pelo desempenho eficiente. Veja algumas características dessa linguagem:

- *orientada a objetos e funcional;*
- *moderna e fácil de aprender;*
- *alta segurança e desempenho;*
- *suporte para programação paralela.*

7. Java

A **Java** é uma linguagem versátil e encontra aplicação em vários contextos, desde o desenvolvimento web até sistemas distribuídos. Veja algumas das suas peculiaridades:

- *portabilidade e versatilidade;*
- *larga utilização no ambiente corporativo;*
- *forte presença em sistemas de back-end e middleware.*

MÓDULO I

8. Go

A **Go** — ou **Golang** —, criada pelo Google, enfatiza simplicidade, eficiência e concorrência, com uma excelente compilação. As características mais marcantes da Go são:

- *ênfase na concorrência;*
- *utilização para um desenvolvimento eficiente;*
- *ideal para sistemas distribuídos e aplicações em larga escala;*
- *compilação rápida, facilitando o desenvolvimento ágil.*

9. SQL

A **SQL** (*Structured Query Language*) é uma linguagem de programação para o gerenciamento de bancos de dados relacionais, sendo uma alternativa mais usada em sistemas de gestão de bancos de dados. Algumas das suas características incluem:

- *manipulação e consulta de dados;*
- *fundamentação de sistemas de gerenciamento de bancos de dados;*
- *grande poder para consultas complexas e relatórios;*
- *uso essencial para profissionais de banco de dados e de análise de dados.*

MÓDULO I

10. Ruby

Por fim, a Ruby, que é conhecida por sua simplicidade e pela produtividade, é usada, principalmente, no desenvolvimento web. Algumas características marcantes da linguagem abarcam:

- sintaxe elegante e intuitiva;
- foco em desenvolvimento ágil;
- comunidade ativa e forte suporte à automação de tarefas.

Buscar a linguagem de programação ideal — especialmente quando falamos sobre linguagem de programação para iniciantes — é uma tarefa muito importante para os desenvolvedores, pois cada projeto envolve requisitos específicos e que demandam escolhas estratégicas.

MÓDULO I

A seguir, trouxemos mais algumas dicas bem valiosas para orientá-lo nessa jornada:

- **JavaScript no front-end:** para dar vida e dinamismo às interfaces de usuário (*ou o que o usuário vê na tela quando navega em uma página da web, como esta aqui*), a **JavaScript** é a melhor escolha. Então, dominar essa linguagem é essencial para criar experiências web envolventes e responsivas;
- **Comunicação eficiente no back-end: Python, PHP, Go e Ruby** desempenham papéis fundamentais na comunicação entre bancos de dados e aplicações no back-end. No caso, cada uma oferece recursos específicos, adaptando-se às necessidades de diferentes projetos, representando excelentes alternativas à sua escolha;
- **Lidando com dados - SQL:** quando a questão envolve o tratamento de dados, a SQL é a melhor pedida. A sua habilidade em lidar com bancos de dados e executar consultas eficientes é indispensável para o gerenciamento de dados em aplicações web;

MÓDULO I

- **Destaque no desenvolvimento de Web Apps:** em se tratando de desenvolvimento de **Web Apps**, **C#**, **JavaScript**, **Java**, **Go** e **Ruby** são ótimas opções, seja pela robustez da **C#**, seja pela versatilidade da **JavaScript**, seja pela escalabilidade da **Java**, seja pela eficiência da **Go**, seja até mesmo pela elegância da **Ruby**. Na verdade, a sua escolha vai sempre depender das demandas específicas do seu projeto;
- **Desenvolvimento de jogos:** quando o assunto diz respeito a jogos, por outro lado, opte por linguagens como **C++**, **C#**, **JavaScript** e **Java**, que são ideais para criar experiências interativas;
- **Aplicativos móveis:** a **C++** e a **Java** se destacam no desenvolvimento de aplicativos móveis, oferecendo um alto desempenho. No entanto, como vimos, para produtos iOS, a linguagem **Swift** é a melhor escolha;
- **Processamento de dados:** em cenários que exigem computação estatística e processamento de dados, **Python**, **SQL** e **Ruby** se destacam, proporcionando ferramentas excelentes para a análise e a manipulação de informações, portanto, são as melhores pedidas;

MÓDULO I

- Versatilidade e popularidade: a **C++** é reconhecida como a linguagem mais versátil da lista, embora a **Java** também seja altamente flexível. Quanto à popularidade, saiba que a **Python** lidera no desenvolvimento web, seguida por **Java**, **JavaScript**, **C++** e **C#**.
- Depois de conhecer as principais linguagens de programação, ficou claro para você que cada uma oferece características únicas e adaptáveis a diferentes contextos? A escolha da linguagem certa depende não só da tarefa em mãos, mas também das preferências e da familiaridade do desenvolvedor, combinado?!

MÓDULO I

JavaScript não é Java

A primeira coisa que você precisa saber: **JavaScript** não tem nada a ver com **Java**. **Java** é uma linguagem **server-side**, como **PHP**, **Ruby**, **Python** e tantas outras. A única coisa parecida entre eles é o nome.

Sabendo disso, quero que saiba que **JavaScript** é uma linguagem de programação client-side.

Ela é utilizada para controlar o **HTML** e o **CSS** para manipular comportamentos na página.

Me pergunte agora: "Como assim comportamento?". Agora eu respondo: um comportamento comum, por exemplo, é um submenu. Sabe quando você passa o mouse em um item do menu, e aparece um submenu com vários outros itens? Pois é. A obrigação de fazer aparecer esse submenu é do **JavaScript**.

O submenu estava escondido, e quando passamos o mouse no item, o submenu aparece.

Todo esse comportamento quem vai executar é o **JavaScript**.

MÓDULO I

Camada de comportamento

Você já deve ter lido a parte que fala sobre o desenvolvimento separando em camadas, onde explicamos que existem três camadas básicas no desenvolvimento para Web: a informação que fica com o **HTML**, a formatação, que fica com o **CSS** e o comportamento, que fica com o **JavaScript**.

O **JavaScript** é a terceira camada de desenvolvimento por que ele manipula as duas primeiras camadas, isto é: **HTML** e **CSS**. Imagine que você precise de um Slider de imagens. Toda a movimentação, ações de cliques nas setinhas e etc, é o **JavaScript** que vai cuidar. É isso que chamamos de comportamento.

MÓDULO I

Como relacionar um arquivo **JavaScript** no seu HTML?

Para adicionar códigos **JavaScript** à uma página devemos usar a *tag* **<script>**, podendo fazer essa inserção de duas formas:

1. Inserindo códigos na própria página (*inline*):

Cria-se uma *tag* **<script>**, informando que o valor do atributo '**type**' é '**text/javascript**', então, coloca-se o código **JavaScript** dentro dessa *tag*.

Exemplo:

```
1. <script type="text/javascript">
2.     alert('Olá mundo!');
3. </script>
```

MÓDULO I

2. Relacionando um arquivo externo na página:

Essa forma é bem parecida com a inserção de códigos **JavaScript** *inline*, a maior diferença é que não coloca-se o código **JavaScript** dentro da *tag*, visto que esse código estará em um arquivo externo. Assim, simplesmente é preenchido o atributo '**src**' da *tag* **<script>** com o caminho para o arquivo em questão. Essa forma também permite carregar arquivos **JavaScript** sem ter que baixá-los para o seu projeto. Isso é bastante utilizado como uma forma de fazer com que arquivos que são usados por muitos projetos, como por exemplo a **jQuery** (framework **JavaScript**), fiquem armazenados em cache, sendo então carregados de forma mais rápida.

MÓDULO I

Exemplo 1 - adicionando um JavaScript do nosso projeto:

Imagine que o projeto está com a seguinte estrutura de diretórios:

```
projeto/  
  index.html (página que irá adicionar o arquivo JavaScript)  
  js/  
    meu-arquivo.js
```

Assim, se queremos que a página `index.html` carregue o arquivo `js/meu-arquivo.js`, utilizamos a seguinte marcação:

```
<script type="text/javascript" src="js/meu-arquivo.js"></script>
```

MÓDULO I

Exemplo 2 - adicionando um JavaScript da internet:

Nesse exemplo, é carregado o *framework* **JavaScript** jQuery disponibilizado pela Google por um serviço de hospedagem de *libraries* (bibliotecas) **JavaScript**. Disponível em:

<https://developers.google.com/speed/libraries/devguide?hl=pt-br#jquery>:

```
1. <script
2.   type="text/javascript"
3.   src="//ajax.googleapis.com/ajax/libs/jquery/1.10.2/jquery.min.js">
4. </script>
```

MÓDULO I

Qual é o melhor local para colocar a *tag* **<script>**?

No geral, o melhor local para ser inserida uma *tag* **<script>** é antes do fechamento da *tag* **<body>**.

Isso se dá devido ao fato de que o browser, ao encontrar uma *tag* **<script>**, precisa executar o que foi especificado ou dentro da *tag* ou pelo atributo '**src**', bloqueando assim a renderização do restante da página. Assim, o código **JavaScript** é executado assim que é interpretado, logo, se existem elementos abaixo do código em questão que são manipulados por esse código (por exemplo, se quiser remover todos os links de uma determinada página e os links encontram-se abaixo da *tag* **<script>**), é necessário adicionar eventos indicando que a página já foi carregada completamente, caso contrário, é bem possível que o código não funcione corretamente

MÓDULO I

O que são variáveis?

Variável é um dos conceitos mais importantes no estudo de programação, independente da plataforma ou linguagem utilizada. Uma variável refere-se a um espaço na memória do computador utilizado para guardar informações que serão usadas em seus programas.

Para elucidar o conceito, imagine que a memória de seu computador é um armário com 100 gavetas e você guarda cada tipo de objeto em uma gaveta diferente. Provavelmente você vai querer criar etiquetas para referenciar o que guarda em cada gaveta.

Mas por que o nome "variável"?

Porque uma variável pode ter o seu valor alterado durante a execução de um programa. Ainda na analogia anterior, suponha que há uma gaveta "Meias" com 2 meias; se necessário, pode-se adicionar mais 5 meias e, caso seja um colecionador, até 200. Assim a gaveta "Meias" muda o seu valor.

MÓDULO I

Criando variáveis

Para criar uma variável utiliza-se **var** (*opcional*) e, para determinar o seu valor, o operador de atribuição (**=**). Para facilitar a compreensão do código, deve-se sempre escolher um nome que identifique o tipo de dado a ser armazenado.

Exemplo:

```
1. <script type="text/javascript">  
2.   var nome = "Gabriel Mendonça";  
3.   var idade = 25;  
4. </script>
```

MÓDULO I

Note também que no final de cada linha foi utilizado o ponto-e-vírgula (**;**), responsável por delimitar *expressões*, *statements* e *construtores*. Embora não seja obrigatório, faz com que o seu código seja mais preciso e claro.

Observação: o código ainda pode ser simplificado deixando apenas uma declaração **var** e separando cada variável por vírgula e fechando a declaração em seu final com ponto-e-vírgula.

Testando:

Abra seu editor de texto preferido; salve o arquivo **HTML** em uma pasta de sua preferência com a estrutura básica, e abra o arquivo em um navegador.

Observação: Em vez de copiar e colar, é interessante que você digite o código para que se adapte à sintaxe da linguagem.

MÓDULO I

Exemplo 1

```
1. <script type="text/javascript">
2.   var nome = prompt('Digite seu nome: ');
3.   alert(nome + ', seja bem vindo!');
4. </script>
```

Nesse exemplo, declaramos a variável "**nome**", onde guardamos o nome que foi solicitado ao usuário através do método **prompt()**. Após o nome informado ser armazenado (isso é, se tornado valor da variável), uma mensagem de boas vindas é apresentada ao usuário através do método **alert()** utilizando o nome armazenado na variável.

MÓDULO I

Exemplo 2

Nesse exemplo, vimos algumas coisas novas: operadores aritméticos de soma e subtração (+, -), o operador de concatenação (+, para operar strings) e de comentários (// para linha única e /* */ para múltiplas linhas). Um comentário é um trecho no código que não é executado, e por isso serve como um espaço para explicações e descrições relacionadas ao código ou até mesmo para evitar a execução de um bloco de código.

```
1. <script type="text/javascript">
2.   /* Este é um script para cálculo de idade! */
3.
4.   // Declara o ano atual para fazer o cálculo
5.   var anoAtual = 2014;
6.
7.   // Pede que o usuário digite o ano em que nasceu
8.   var anoNascimento = prompt('Digite o ano em que você nasceu.');
```

```
9.
10.  // Calcula a idade do usuário e armazena na variável idade
11.  var idade = anoAtual - anoNascimento;
12.
13.  // Mostra ao usuário a idade que ele possui
14.  alert("Sua idade é: " + idade + " anos");
15. </script>
```

MÓDULO I

Definindo nomes de variáveis

Quando criamos variáveis temos de levar em consideração algumas regras específicas da linguagem:

- **Uma variável é *case-sensitive***
Isso significa que nomes com letras maiúsculas são diferentes de nomes com letras minúsculas: para um programa em JavaScript, **Nome** é diferente de **nome**.

Caracteres válidos

- **Palavras:** embora a maioria dos navegadores já reconheça uma variedade de caracteres UTF-8 (como palavras com acentos e "ø_ø", por exemplo), o recomendado é o uso apenas letras de MAIÚSCULAS e minúsculas, sem acentos e espaços.

```
var meuNOME = "joão";
```

```
var meunome = "jósé";
```

```
var MeuNoMe = "Thiago";
```

```
var ø_ø/ ção = "JavaScript é legal!"; // Não recomendado, entretanto
```

MÓDULO I

- **Números:** desde que sejam precedidos por uma ou mais letras.

```
var camisa9 = "ronaldo";
```

```
var camisa23olateral = "joaozinho";
```

- **Underline e cifrão:** "_" e "\$" também são permitidos em qualquer posição, mas pouco usados.

```
var _nome = 'Gabriel'
```

```
var segundo_nome = "Mendonça"
```

```
var $ultimo_nome_ = ""
```

MÓDULO I

Nome reservados pela linguagem

Alguns nomes não podem ser utilizados para criação de variáveis pois estão reservados de alguma forma à linguagem. São eles:

abstract, boolean, break, byte, case, catch, char, class, const, continue, default, do, double, else, extends, false, final, finally, float, for, function, goto, if, implements, import, in, instanceof, int, interface, long, native, new, null, package, private, protected, public, return, short, static, super, switch, synchronized, this, throw, throws, transient, true, try, var, void, while, with.

MÓDULO I

Tipos de variáveis

Quando falamos em tipos de variáveis, temos as linguagens chamadas de **fortemente** tipadas e **fracamente** tipadas.

Em linguagens **fortemente** tipadas, definimos o tipo da variável no momento de sua criação.

Exemplos de linguagens do tipo: **Java; C; C++.**

Em linguagens **fracamente tipadas**, não precisamos definir o tipo da variável, ela é tipada automaticamente quando recebe um valor.

Exemplos de linguagens do tipo: **Python; Ruby; Javascript.**

Float - Variáveis com ponto flutuante ou casas decimais.

```
var peso = 32.59345;
```

```
var PI = 3.14;
```

```
var meu_saldo = -1034.32
```

MÓDULO I

- **String** - Variáveis de texto, normalmente chamada de "cadeia de caracteres".

Os valores desse tipo são atribuídos utilizando aspas duplas (") ou aspas simples (') como delimitador.

```
var nome = "Gabriel Mendonça";
```

```
var data_nascimento = "17 de Junho de 1988";
```

```
var email = "gabriel@host2.com.br";
```

```
var tempo = "20s";
```

Observação: Tudo o que é declarado entre os delimitadores (") ou (') é entendido como parte da string, mesmo que sejam números.

MÓDULO I

- **Booleanos** - Tipo de dado de dois valores: "**true**" (**verdadeiro**) ou "**false**" (**falso**).

```
var verdadeiro = true;  
var verdadeiro2 = 1;  
var falso1 = false;  
var false2 = 0;  
var falso3 = null
```

Observação: Apesar dos valores **true** e **false** representarem, respectivamente, os valores "verdadeiro" e "falso", pode-se utilizar outros valores para essa representação, como exemplificado acima.

- **Int** - Variáveis com valores inteiros.

```
var idade = 17;  
var graus = -3;  
var pontos = 0;  
var numeroGrande = 2000009283;
```


MÓDULO I

- **Arrays** - Um **array** referencia a vários espaços na memória.

É um conjunto de valores e/ou variáveis organizadas por índice (que pode ser um valor **inteiro** ou **string**).

O entendimento desse tipo é muito importante.

Retomando a analogia inicial: imagine que em uma gaveta você queira armazenar bebidas. Nela teria *suco*, *refrigerante*, *café*, *água mineral*... Perceba que esses itens formam uma lista e que todos os itens são bebidas, ou seja, itens de uma mesma categoria. Com o uso de arrays evitamos declarar diversas variáveis para um mesmo grupo (nada de **bebida1**, **bebida2**!), acessando os itens pelos seus respectivos índices:

bebida[1], **bebida[2]**. Se você perceber, declarar várias variáveis para uma mesma coisa usaria desnecessariamente várias gavetas, enquanto declarar um array para armazenar todas as "mesmas coisas" usaria apenas uma. Não soa mais inteligente? :-)

Pense que, enquanto uma variável é uma casa, um array é um prédio; ou que um array é como um gaveteiro.

MÓDULO I

Operadores

Os operadores vão nos permitir fazer operações(mesmo!? Não me diga...) matemáticas, de comparação e lógicas

Aritméticos

Para as operações matemáticas básicas são utilizados os seguintes, adição(+), subtração(-), multiplicação(*) e divisão(/).

Você notou que podemos ter resultados com casas decimais e que é retornada a constante **Infinity** em qualquer número dividido por zero.

```
1. //Adição
2. 2+2 //4
3. 2.3+4 //6.3
4. 1.5+1.5 //3
5.
6. //Subtração
7. 2-2 //0
8. 8-5 //-8
9. 3.2-1 //2.2
10.
11. //Multiplicação
12. 2*3 //6
13. 1.5*2 //3
14.
15. //Divisão
16. 1/2 //0.5
17. 1.5/2 //0.75
18. 2/0 //Infinity
```

MÓDULO I

Decremento (--)

O comportamento desse operador é parecido com o de incremento (acho que você já entendeu). Ele subtrai um da variável. Se utilizado antes (--x) subtrai um e retorna o valor, caso o operador seja utilizado depois da variável (x--) retorna o valor e subtrai um.

```
1. var x = 2;  
2. --x //1  
3. x-- //1  
4.
```

Incremento (++)

Adiciona uma variável. Se utilizado antes (++x) adiciona um e retorna o valor, caso o operador seja utilizado depois da variável (x++) retorna o valor e adiciona um.

```
1. var x = 1;  
2. ++x //2  
3. x++ //2
```

Resto (%)

Retorna o resto inteiro da divisão.

```
1. 5%4 //1  
2. 4%5 //4  
3.
```

MÓDULO I

Igual (==)

Retorna verdadeiro se os valores comparados forem iguais.

```
1. 1=='1' //true
2.
```

De comparação

Igual estrito (===)

Esse operador é mais severo, só retorna verdadeiro se o valor e o tipo comparados forem iguais.

```
1. 3==='3' //false
2. 3===3 //true
3.
```

Não igual/ Diferente de (!=)

Retorna verdadeiro se os valores comparados não forem iguais. Esse operador também pode ser chamado de diferente de.

```
1. 4!=1 //true
2.
```

Não igual estrito (!==)

Não se engane, esse operador vai retornar verdadeiro se os valores e ou os tipos forem diferentes.

```
1. 3!== '3' //true
2. 3!== 3 //false
3. 3!== 4 //true
```

MÓDULO I

Maior que (>)

Compara se o operador da esquerda é maior que o da direita. Caso seja retorna verdadeiro

```
1. 1>2 //false
2. 5>3 //true
3. 4>'1' //true
```

Maior ou igual que (>=)

Compara se o operador da esquerda é maior ou igual ao da direita. Caso seja retorna verdadeiro

```
1. 1>=2 //false
2. 5>=3 //true
3. 4>='1' //true
4. 2>=2 // true
```

Menor que (<)

Compara se o operador da esquerda é menor que o da direita. Caso seja retorna verdadeiro

```
1. 1<2 //true
2. 5<3 //false
3. 4<'1' //false
```

Menor ou igual que (<=)

Compara se o operador da esquerda é menor ou igual ao da direita. Caso seja retorna verdadeiro

```
1. 1<=2 //true
2. 5<=3 //false
3. 4<='1' //false
4. 2<=2 // true
```

MÓDULO I

Arrays

Valores agrupados

Com o **Array** é possível armazenar um conjunto de quaisquer valores **javascript**, como **números**, **caracteres** ou **textos** ou uma mistura deles. Imagine o **array** como um gaveteiro onde você pode adicionar ou retirar gavetas e cada gaveta contém o objeto que quiser, vamos criar aqui um gaveteiro onde a primeira gaveta contém o valor **10** a segunda **20** e a terceira **30**.

```
1. var gaveteiro = [10,20,30];
```

Simples não é? A diferença no **Javascript** é que a contagem de cada posição do **array** começa em **zero**, assim temos: gaveta **0**, gaveta **1** e gaveta **2**.

MÓDULO I

Acessando elementos do array

Com o nosso array criado podemos visualizar cada uma das posições individualmente colocando o índice dentro de colchetes:

```
1. console.log(gaveteiro[2]); //30
2. console.log(gaveteiro[1]); //20
3. console.log(gaveteiro[0]); //10
```

Também podemos alterar o valor de cada posição da seguinte forma:

```
1. var gaveteiro = [10,20,30];
2. gaveteiro[2] = 99;
3. console.log(gaveteiro[2]);
```

Assim dizemos que gaveteiro na posição 2 recebeu o valor 99.

MÓDULO I

Adicionando elementos no array

Caso precise **adicionar** uma nova gaveta, podemos usar o método ***push***:

```
1. var gaveteiro = [10,20,30];  
2. gaveteiro.push(100);  
3. console.log(gaveteiro[3]); //100
```

O método ***push*** recebe **100** como parametro e adiciona na ultima posição do **array**.

Removendo elementos no array

Caso precise **remover** uma gaveta(**array**), podemos usar os seguintes métodos:

- Para **remover** a ultima gaveta, utilizamos o ***pop***:

```
1. var gaveteiro = [10,20,30];  
2. console.log(gaveteiro[2]); //30  
3. gaveteiro.pop();  
4. console.log(gaveteiro[2]); //undefined
```


MÓDULO I

- Para **remover** a primeira gaveta, utilizamos o **shift**:

```
1. var gaveteiro = [10,20,30];  
2. console.log(gaveteiro[0]); //10  
3. gaveteiro.shift();  
4. console.log(gaveteiro[0]); //20
```

- Para **retornar** apenas algumas gavetas (**recortar**), utilizamos o **slice**:

```
1. var gaveteiro = [10,20,30];  
2. var novaGaveta = gaveteiro.slice(1,3);  
3. console.log(novaGaveta); //[20, 30]
```

MÓDULO I

Quantidade de elementos do array

Depois de ter adicionado várias gavetas, pode surgir a necessidade de saber quantas já existem, para isso vamos acessar a propriedade ***length***:

```
1. var gaveteiro = [1,2,3,10,20,30];
2. console.log(gaveteiro.length); //6
3. gaveteiro.push(100);
4. gaveteiro.push(200);
5. gaveteiro.push(300);
6. gaveteiro.push(400);
7. console.log(gaveteiro.length); //10
8. gaveteiro.push(200);
9. console.log(gaveteiro.length); //11
```

MÓDULO I

O que é um Framework? Entendendo o conceito

Em uma busca rápida na web, é possível encontrar diversas definições para **Framework**, algumas simples, outras mais elaboradas, mas o ponto comum entre todas é a **reusabilidade**. Assim um Framework tem como principal objetivo resolver problemas recorrentes com uma abordagem genérica, permitindo ao desenvolvedor focar seus esforços na resolução do problema em si, e não ficar reescrevendo software. Você pode se perguntar, então **Framework** é uma biblioteca? Bem quase isso, pode-se dizer que é um conjunto de bibliotecas ou componentes que são usados para criar uma base onde sua aplicação será construída.

As **frameworks** ajudam no desenvolvimento rápido e seguro de aplicações mas é recomendável, estudar antes a tecnologia em que a mesma é desenvolvida. Logo é importante estudar os aspectos básicos do **javascript** antes de se aventurar em uma **framework** da Linguagem. Possuindo o conhecimento das tecnologias da **Framework** é possível fazer suas próprias modificações para que a **framework** possa atender todas as necessidades do desenvolvedor.

MÓDULO I

Frameworks em Destaque no Desenvolvimento Web

Existe diversas frameworks para desenvolvimento de aplicações, mas o nosso foco será em ferramentas para criação de sites. Abaixo são listados alguns que podem ajudar na sua carreira.

Bootstrap

É utilizado para desenvolvimento de componentes de interface de sites, utilizando **html**, **css** e **javascript** foi desenvolvido levando em consideração técnicas de design, para melhorar a experiência visual do usuário.

MÓDULO I

O que é jQuery?

jQuery, segundo definição consta em seu site, trata-se de uma rápida, pequena e rica em *features* biblioteca **JavaScript**.

Como o lema "***Write less, do more.***" (escreva menos, faça mais), **jQuery** revolucionou o desenvolvimento web, sendo presente em inúmeros projetos atualmente.

Mas, por que é tão utilizado?

Exatamente pelo seu lema. Com **jQuery** é possível fazer diversos efeitos com poucas linhas e, que custariam dezenas de linhas em **JavaScript** puro.

Alguns recursos oferecidos facilmente pelo **jQuery**:

- Seleção e manipulação de elementos **HTML**
- Manipulação de **CSS**
- Efeitos e animações
- *Navegação* pelo DOM
- Ajax
- Eventos

MÓDULO I

Como uso?

Igualmente qualquer arquivo **JavaScript** você precisa inseri-lo na página com as tags **script**. Você pode fazer o link para uma CDN (Content Delivery Network) como a do Google ou fazer o download do **framework** no seu projeto e linkar direto (vale lembrar que se optar por essa opção, faça o link para a versão **minificada**).

```
1. <script src="js/jquery.min.js"></script>
```

A sintaxe

A sintaxe básica é a seguinte:

```
1. $('seletorHTML').acao();
```

MÓDULO I

Manipulação HTML

```
1. $( "#botao" ).html( "Sucesso!" )
```

No exemplo acima, selecionamos o elemento com *ID* botão e inserimos a *string* "Sucesso!".

Eventos

```
1. var box = $( "#box" );  
2. $( "#botao" ).on( "click", function( event ) {  
3.     box.show();  
4. });
```

No **snippet** acima, guardamos em uma variável o elemento com *ID* box. Em seguida, adicionamos ao elemento com *ID* 'botao' o evento do *clique*. Ao ser disparado, esse evento mostra o elemento, que guardamos na variável box anteriormente, através do método **show()**.

MÓDULO I

Explicação linha por linha:

1. **var box = \$("#box");**

Aqui, estamos selecionando um elemento HTML com o ID "box" (por exemplo, uma <div id="box">) e guardando ele em uma variável chamada box para usar depois.

2. **\$("#botao").on("click", function(event) {**

Selecionamos um elemento com o ID "botao" (por exemplo, um <button id="botao">) e dizemos que, quando ele for clicado ("click"), a função dentro dele será executada.

3. **box.show();**

Quando o botão for clicado, o elemento box (que selecionamos na linha 1) será exibido na tela (se estiver oculto). O .show() é um método do jQuery que faz isso.

4. **});**

Fechamos a função do clique e o evento.

MÓDULO I

Resumo:

- **O que o código faz?**

Quando você clica no botão (**#botao**), ele mostra (**show()**) a caixa (**#box**) na tela.

- **Exemplo prático:**

Se você tiver uma `<div id="box" style="display: none;">Olá!</div>` escondida e um `<button id="botao">Clique aqui</button>`, ao clicar no botão, a mensagem "Olá!" aparecerá.

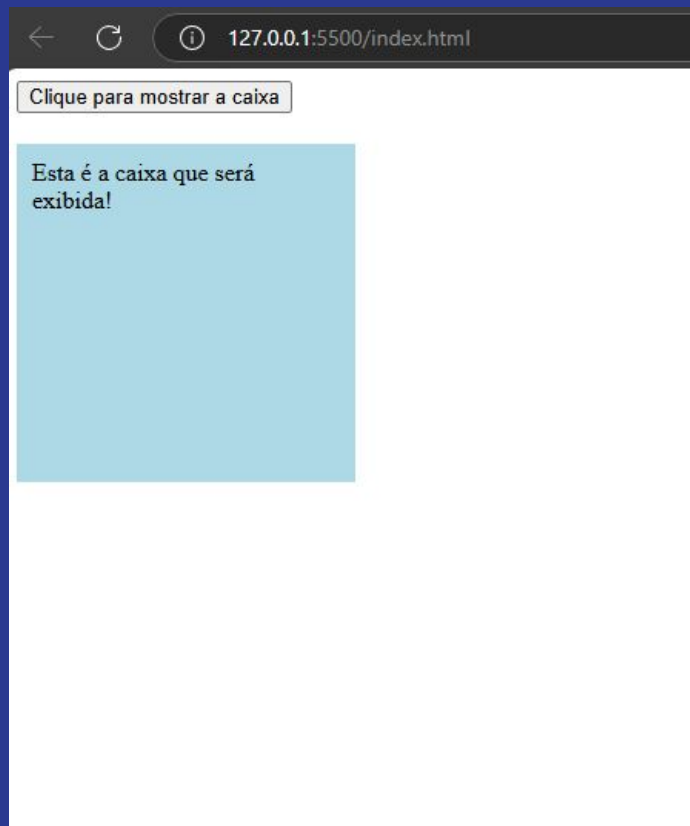
MÓDULO I

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>Teste do Código jQuery</title>
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
  <style>
    #box {
      width: 200px;
      height: 200px;
      background-color: lightblue;
      display: none; /* Inicia oculto */
      margin-top: 20px;
      padding: 10px;
    }
  </style>
</head>
<body>

  <button id="botao">Clique para mostrar a caixa</button>
  <div id="box">Esta é a caixa que será exibida!</div>

  <script>
    var box = $("#box");
    $("#botao").on("click", function(event) {
      box.show();
    });
  </script>

</body>
</html>
```



MÓDULO I

EXERCÍCIOS...

MÓDULO I

FIM...